

## CASE STUDY : SERVERLESS EVENT-DRIVEN AUTOMATION WITH AWS

Duration : 5 Hours

### 1. Project Description

#### 1.1. Background

In the era of modern cloud computing, serverless and event-driven architectures have become optimal solutions for building applications that are highly scalable, cost-effective, and easy to manage. This project aims to implement a fully serverless automated order processing system using the latest AWS services, eliminating the complexity associated with traditional infrastructure management. By leveraging managed cloud services, the system can automatically scale based on demand while ensuring high availability and operational efficiency.

Through the adoption of an event-driven approach, every stage of the order lifecycle is triggered by events, enabling seamless integration between services and real-time processing. This architecture allows organizations to focus on business logic rather than infrastructure concerns, reduce operational overhead, and accelerate application development. As a result, the proposed solution provides a robust, flexible, and future-ready platform for handling automated order processing in a cloud-native environment.

#### 1.2. Task

As a Cloud Engineer, you are tasked with building a complete serverless order processing system with the following specifications:

##### TASK LIST:

1. Log in to the AWS Management Console using your class account.
2. Set up the infrastructure using CloudFormation.
3. Configure the VPC according to the specified requirements.
4. Set up Amazon S3 for file storage and static assets.
5. Deploy AWS Lambda functions to handle business logic processing.
6. Configure Amazon RDS for data persistence.
7. Create AWS Step Functions for workflow orchestration.
8. Set up Amazon EventBridge for event-driven automation.
9. Configure Amazon API Gateway to expose RESTful endpoints.
10. Deploy the frontend application using AWS Amplify.
11. Implement a CI/CD pipeline using GitHub Actions.
12. Verify that the application is accessible from the public network and that all features function correctly.

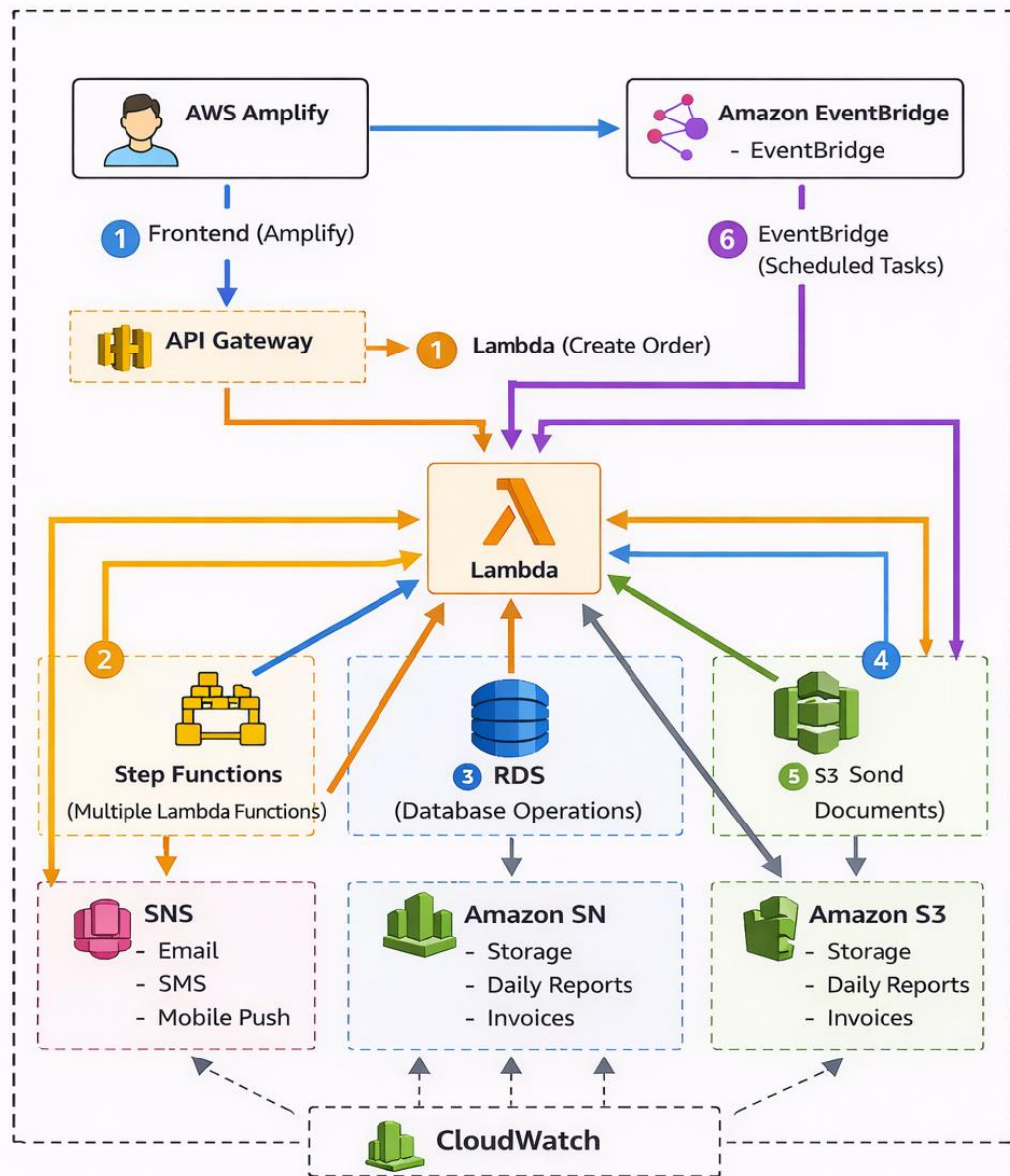
#### 1.3. Technical Details

1. Region default: Virginia (us-east-1)
2. Service prefix: **All service names must begin with lks-**
3. Source code repository: <https://github.com/handipradana/wakawaka.git>
4. OS for EC2: Amazon Linux 2023 (if needed)
5. Systems can handle asynchronous order processing with email notifications
6. Runtime Lambda: Python 3.11
7. Database: PostgreSQL 15.x

## 1.4. References

1. References for CloudFormation  
<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html>
2. References for VPC <https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html>
3. References for S3  
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>
4. References for Lambda  
<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
5. References for RDS  
<https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>
6. References for Step Function <https://docs.aws.amazon.com/step-functions/latest/dg/welcome.html>
7. References for EventBridge  
<https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-what-is.html>
8. References for API Gateway  
<https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>
9. References for Amplify  
<https://docs.aws.amazon.com/amplify/latest/userguide/welcome.html>
10. References for GitHub Personal Access Token  
<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>
11. References for GitHub Action  
<https://docs.github.com/en/actions/get-started/quickstart>

## 2. ARCHITECTURE DIAGRAM



1. User → Frontend (Amplify) → API Gateway → Lambda (Create Order)
2. Lambda → Step Functions (Multiple Lambda Functions)
3. Lambda → RDS (Database Operations)
4. Lambda → S3 (Store Documents)
5. Lambda → SNS (Send Notifications)
6. EventBridge → Lambda (Scheduled Tasks)
7. CloudWatch ← All Services (Monitoring)

### 3. Implementation using CloudFormation

Use **CloudFormation** to provision and manage **infrastructure components**, including (but not limited to) networking resources, Amazon S3 buckets, Amazon RDS, AWS Lambda functions, Simple Notifications Service (SNS) and API Gateway, ensuring the entire environment is defined as Infrastructure as Code (IaC).

#### 1. Networking and Security

- Virtual Private Cloud (VPC)

You need to configure a Virtual Private Cloud (VPC) with a CIDR block of 20.1.0.0/20 and an IPv6 /56 range provided by Amazon, give the name is lks-vpc-serverless. The VPC should include two public subnets with IPv4 CIDR ranges 20.1.0.0/25 and 20.1.1.0/25, as well as two private subnets with IPv4 CIDR ranges 20.1.11.0/25 and 20.1.12.0/25. All subnets must be using IPV6 /64 and enable Egress only internet for IPV6. Additionally, you must create two route tables: one for public subnets and one for private subnets.

- Security Group

You are required to configure Security Groups to control inbound traffic for different components of the infrastructure, follow this instruction:

1. Create a Security Group, give named is **lks-sg-lambda**, the description is security for lambda function, no need inbound.
2. Create a Security Group, give named is **lks-sg-rds**, the description of security for RDS and open port 5432 from lks-sg-lambda.
3. Create a Security Group, give named is **lks-sg-vpc-endpoint**, the description of security for vpc endpoint, open port HTTP and HTTPS from CIDR 20.1.0.0/20

- VPC Endpoint

You need create vpc endpoint for this service:

- S3 Gateway Endpoint (com.amazonaws.us-east-1.s3) give name is **lks-s3-endpoints**.
- EventBridge Interface Endpoint (com.amazonaws.us-east-1.events) give name is **lks-eventbridge-endpoints**.
- Step Functions Interface Endpoint (com.amazonaws.us-east-1.states) give name is **lks-steps-endpoints**.
- SNS Interface Endpoint (com.amazonaws.us-east-1.sns) give name is **lks-sns-endpoints**.

#### 2. S3 Bucket

You need to create a S3 bucket, the function bucket is order Documents and Application Logs. Follow the instructions on below:

- S3 Bucket Order Documents

Give the name is lks-orders-yourname-2026, enabled the versioning and setting the lifecycle rules for transition to intelligent-tiering after 30 days, transition to glacier after 90 days and delete permanently after 365 days. Setting the bucket policy allow to all and setting the CORS to allow frontend access.

- S3 Bucket Application logs  
Give the name is lks-logs-yourname-2026, setting the lifecycle rules for transition to intelligent-tiering after 7 days, delete permanently after 90 days.

### 3. Relation Database Service (RDS)

The **database configuration** is designed to support the Order Management system using a managed relational database service on AWS. The database engine used is **PostgreSQL version 15.x**, ensuring high performance, reliability, and compatibility with modern SQL features. The DB instance is identified as **lks-rds-orders** and runs on the **db.t3.micro** instance class, which is suitable for lightweight workloads and development or training environments. Storage is provisioned at **20 GB** using the **gp3** volume type to provide consistent performance and cost efficiency.

From a networking and security perspective, the database is deployed within the **lks-vpc-serverless** VPC and configured to use **private subnets only**, ensuring that the database is not directly exposed to the public internet. **Public access is disabled**, and access control is enforced through the **lks-sg-rds security group**, which restricts inbound connections to trusted resources such as Lambda functions running within the same VPC. The database is created with the name **ordersdb**, using **dbadmin** as the master username and a predefined master password **TechnoCloud2026!**. To support data protection and recovery, automated backups are enabled with a **retention period of 7 days**.

For **database schema initialization**, a dedicated Lambda function named **lks-lambda-init-db** is used. This function is responsible for automatically creating the required database structures, including the **customers**, **orders**, **order\_items**, and **inventory** tables. In addition, it defines the necessary **indexes and constraints** to maintain data integrity and optimize query performance. Optionally, the function can also insert **sample data** to facilitate testing and validation of the Order Management system during initial deployment.

### 4. Lambda Function

**Lambda Functions** focuses on deploying six AWS Lambda functions, all implemented using **Python 3.11**, to support the end-to-end Order Management system. Each function is configured with specific memory and timeout settings based on its responsibility, ensuring efficient resource utilization and reliable execution across the workflow.

1. The first function, **Order Management (lks-lambda-order-management)**, serves as the core business logic handler for order processing. It is allocated **512 MB of memory** with a **30-second timeout** and is attached to **private subnets within a VPC** to securely access backend resources such as Amazon RDS. This function uses environment variables to configure database connectivity (RDS endpoint, database name, username, and password), define the target Amazon S3 bucket for order-related storage, and reference the AWS Step Functions state machine ARN for orchestration.
2. The second function, **Process Payment (lks-lambda-process-payment)**, is responsible for handling payment-related logic. It is configured with **512 MB of**

**memory** and a **30-second timeout**, providing sufficient capacity to interact with external payment services or perform transaction validation as part of the order lifecycle.

3. The third function, **Update Inventory (lks-lambda-update-inventory)**, manages stock updates after an order is processed. This function is optimized for lightweight operations with **256 MB of memory** and a **45-second timeout**, ensuring fast and efficient inventory adjustments in the database.
4. The fourth function, **Send Notification (lks-lambda-send-notification)**, handles customer or system notifications such as order confirmations or status updates. It is configured with **256 MB of memory** and a **1-minute timeout**, reflecting its simple and time-sensitive execution requirements.
5. The fifth function, **Generate Report (lks-lambda-generate-report)**, is designed for more resource-intensive tasks such as data aggregation and report generation. To support these operations, it is allocated **1024 MB of memory** with a **60-second timeout**, allowing it to process larger datasets and generate reports in formats such as Excel.
6. The sixth function, **Init Database (lks-lambda-init-db)**, is used during initial deployment or maintenance phases to initialize the database schema. It is configured with **512 MB of memory** and an extended **300-second timeout**, enabling it to create tables, indexes, constraints, and optionally insert sample data without interruption.

To standardize dependencies and reduce package duplication, a shared **Lambda Layer** named **lks-layer-dependencies** is created and attached to all Lambda functions. This layer includes commonly used libraries such as **psycopg2-binary**, **boto3**, **requests**, **pandas**, and **openpyxl**, ensuring consistent runtime environments and simplifying maintenance across the entire serverless application.

## 5. Simple Notification Service (SNS)

**Setup Amazon SNS** focuses on configuring Amazon Simple Notification Service (SNS) to deliver real-time notifications related to the Order Management system. An SNS topic named **lks-sns-order-notifications** is created as a centralized communication channel for distributing messages to multiple subscribers in a scalable and reliable manner.

The SNS topic is configured with **email subscriptions**, including **handi@seamolec.org**, allowing key stakeholders and system administrators to receive **important** notifications directly in their inboxes. Once subscribed and confirmed, these recipients will be automatically notified whenever relevant events occur within the system.

The notification types published to this SNS topic include **order confirmations**, **payment status updates**, **inventory alerts**, **system error notifications**, and **report completion messages**. By integrating SNS with Lambda functions and other AWS services, the system ensures timely delivery of critical information, improves operational awareness, and enables faster response to issues or business events.

## 6. API Gateway

**API Gateway** covers the setup of Amazon API Gateway as the main entry point for external clients to interact with the Order Management system. The REST API is named **lks-api-orders** and is configured with a **regional endpoint type, IP address dualstack, and security policy 1.3**, allowing low-latency access within the selected AWS region while maintaining high availability. The API is deployed to a stage named **production**, which represents the stable, live environment for consuming applications.

Several RESTful endpoints are defined to support full order lifecycle management.

|        |              |                       |                             |
|--------|--------------|-----------------------|-----------------------------|
| GET    | /orders      | List Orders           | lks-lambda-order-management |
| POST   | /orders      | Create New Order      | lks-lambda-order-management |
| GET    | /orders/{id} | Get order details     | lks-lambda-order-management |
| PUT    | /orders/{id} | Update order          | lks-lambda-order-management |
| DELETE | /orders/{id} | Delete order          | lks-lambda-order-management |
| GET    | /status{id}  | Check workflow status | lks-lambda-order-management |
| GET    | /customers   | Get Customers         | lks-lambda-order-management |
| GET    | /products    | Get products          | lks-lambda-order-management |

To secure and control API usage, an **API security configuration** is applied using a **Usage Plan** named **lks-usage-plan**. This plan enforces a **rate limit of 1,000 requests per second** with a **burst limit of 2,000 requests**, protecting backend resources from sudden traffic spikes. A **monthly quota of 100,000 requests** is also defined to manage long-term consumption. Access to the API is authenticated using an **API Key** named **lks-api-key**, ensuring that only authorized clients can invoke the API endpoints under the defined usage constraints.

## 4. Implementation without CloudFormation

### 1. Step Function

Step Functions involves defining an AWS Step Functions state machine to orchestrate the end-to-end order processing workflow in a reliable and scalable manner. The state machine is named **lks-stepfunctions-order-workflow** and uses the **Standard** workflow type, which is suitable for long-running, auditable, and fault-tolerant business processes. The IAM role **LabRole** is assigned to this state machine, granting the required permissions to invoke the associated Lambda functions securely.

The workflow begins with the **ValidateOrder** step, which invokes the Lambda function **lks-lambda-order-management** to validate incoming order data, ensure data completeness, and perform initial business rule checks. Once the order is validated, the workflow proceeds to **ProcessPayment**, where the Lambda function **lks-lambda-process-payment** is executed to handle payment authorization and processing.

After the payment step, the workflow enters a **PaymentChoice** state, which evaluates the value of **paymentStatus** returned by the previous Lambda execution.

Based on this condition, the state machine determines the next action, allowing different execution paths for successful payments, failed payments, or pending states.

If the payment is successful, the workflow continues to **UpdateInventory**, invoking the Lambda function **lks-lambda-update-inventory** to adjust stock levels accordingly. Once inventory updates are completed, the state machine transitions to the **InventoryChoice** state, which checks whether the inventory update was successful or failed. If successful, the workflow proceeds to **SendConfirmation**, which triggers the Lambda function **lks-lambda-send-notification** to deliver order confirmation messages or status updates to the customer or relevant systems.

If the payment fails at the **PaymentChoice** state, the workflow transitions to the **PaymentFailed** state, which invokes the Lambda function **lks-lambda-send-notification** to send a payment failure notification to the customer, then continues to the **OrderFailed** state that terminates the workflow with a failed status. Similarly, if the inventory update fails at the **InventoryChoice** state, the workflow transitions to the **InventoryFailed** state, which also invokes the notification Lambda to send a system error notification, before finally proceeding to the **OrderFailed** state.

Throughout the workflow, error handling and fallback paths are defined to gracefully manage failures at each stage. These mechanisms ensure that errors are captured, retries are applied when appropriate, and compensating actions or notifications can be executed, thereby improving the overall resilience and reliability of the order processing system.

## 2. EventBridge

**EventBridge** focuses on configuring Amazon EventBridge rules to enable event-driven automation and scheduled tasks within the Order Management system. These rules allow the system to respond to time-based schedules as well as custom business events in a decoupled and scalable manner.

The first rule, **Daily Report**, is created with the name **lks-eventbridge-daily-report** and uses a scheduled expression of `cron(59 23 * * ? *)`, which triggers the rule **daily at 23:59 UTC**. This rule targets the **lks-lambda-generate-report** Lambda function, enabling the system to automatically generate daily operational or transaction reports without manual intervention.

The second rule, **Order Status Events**, is named **lks-eventbridge-order-status** and is configured using an **event pattern** to capture custom application events related to order status changes (such as order created, paid, shipped, or failed). When such events are published to EventBridge, this rule routes them to the **lks-lambda-send-notification** Lambda function, ensuring that notifications are sent in near real time based on order lifecycle updates.

The third rule, **Low Stock Alert**, is defined with the name **lks-eventbridge-low-stock** and uses a scheduled expression of `rate(1 hour)`, causing it to run every hour. This rule targets a **custom Lambda function** responsible for checking

inventory levels. If low stock conditions are detected, the function can trigger alerts or follow-up actions, helping to maintain inventory availability and prevent stock shortages.

### 3. CloudWatch

**Configure CloudWatch** focuses on implementing monitoring, logging, and alerting to ensure the reliability, performance, and observability of the serverless Order Management system. Amazon CloudWatch is used as the centralized service for collecting logs, tracking metrics, and visualizing system health across all integrated AWS components.

For **CloudWatch Logs**, log groups are created for **all Lambda functions**, as well as for **AWS Step Functions** and **Amazon API Gateway**, ensuring that execution details, errors, and request traces are fully captured. A **log retention policy of 7 days** is applied uniformly to all log groups, balancing operational visibility with cost efficiency and log lifecycle management.

Several **CloudWatch Alarms** are configured to proactively detect abnormal conditions and potential system issues. Alarms are set to trigger when the **Lambda error rate exceeds 5 errors within 5 minutes**, when **API Gateway returns more than 10 client-side (4XX) errors in 5 minutes**, and when **Step Functions experience more than 3 execution failures within 10 minutes**. In addition, an alarm monitors **Amazon RDS CPU utilization**, triggering when usage exceeds **80% for a continuous period of 5 minutes**, helping to identify performance bottlenecks at the database layer.

To provide a unified operational view, a **CloudWatch Dashboard** named **lks-dashboard-serverless** is created. This dashboard includes visual widgets displaying **Lambda performance metrics**, **API Gateway request and error statistics**, **Step Functions execution status**, and **RDS resource metrics**. Together, these monitoring configurations enable real-time visibility, faster troubleshooting, and more effective operational decision-making for the entire system.

### 5. Deploy to Amplify

**Deploy Frontend with AWS Amplify** focuses on delivering a web-based user interface for the Order Management system using a fully managed frontend hosting and CI/CD service. An AWS Amplify application is created with the name **lks-amplify-order-app** and is connected directly to a **GitHub repository**, enabling automatic builds and deployments whenever changes are pushed. The application is configured to deploy from the **main** branch and uses a **Static HTML/CSS/JavaScript** framework, making it lightweight, fast, and easy to maintain. Custom build and deployment steps are defined through a tailored **amplify.yml** file to meet the specific needs of the project.

The frontend application provides several key features to support operational and monitoring activities. It includes a **dashboard with statistical summaries** to give users a quick overview of system performance and order metrics. An **order creation form** is available for submitting new orders, while an **orders management table**

allows users to view, update, and manage existing orders efficiently. The application also features a **reports section** for accessing generated reports, as well as **system status monitoring** to track the health and progress of backend workflows.

To enable seamless integration with backend services, the Amplify application is configured with **environment variables**. The **API\_ENDPOINT** variable points to the Amazon API Gateway URL used by the Order Management API, while **API\_KEY** is used to authenticate requests under the defined usage plan. The **AWS\_REGION** variable is set to **us-east-1**, ensuring that frontend requests are correctly routed to the appropriate AWS region. This configuration allows the frontend to securely and reliably communicate with the serverless backend infrastructure.

## 6. Implement CI/CD Pipeline

CI/CD Pipeline Implementation describes the setup of an automated deployment pipeline using GitHub Actions to ensure consistent, reliable, and repeatable delivery of frontend updates. The CI/CD workflow is defined in the file `.github/workflows/lks-deploy.yml`, which orchestrates deployment and notification steps when changes are made to the frontend codebase. Invite the handipradana account for collaboration.

The workflow is triggered on push or pull\_request events to the master branch for files in the frontend/ directory or workflow configuration changes. It can also be manually triggered via workflow\_dispatch.

The workflow consists of two structured jobs. The deploy-frontend job manages the entire deployment lifecycle to AWS Amplify. It begins by checking out the repository and configuring AWS credentials using GitHub Secrets. The credentials are immediately validated to ensure secure and authenticated access to AWS services. Next, essential frontend files (index.html, app.js, and amplify.yml) are validated for existence. The job then verifies if the target AWS Amplify application exists. If the application is found, it triggers a new Amplify build and deployment job, actively monitoring its status until completion (success, failure, or timeout). Upon successful deployment, it retrieves and outputs the application URL. Finally, it generates a comprehensive deployment summary in the GitHub Actions interface, providing clear visibility into the deployment outcome.

The notify job runs after the deployment, regardless of its success or failure, but only for the main branch. It determines the final deployment status and sends a detailed notification via an AWS SNS topic. This notification includes the deployment status, commit information, author, and a direct link to the workflow run, ensuring the team has complete visibility into the deployment process.

To support secure automation, the workflow requires several GitHub Secrets: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, and `AWS_SESSION_TOKEN` for authenticating deployment actions against AWS services. By implementing this focused CI/CD pipeline, the system achieves automated, reliable, and observable deployments for the frontend component, reducing manual effort and accelerating delivery cycles.

## 7. Testing & Verification

1. **Testing & Verification** focuses on validating that all components of the Order Management system function correctly, both individually and as an integrated solution. This phase ensures that the system meets functional requirements, performs reliably, and behaves as expected under real-world usage scenarios. For **API Testing**, all exposed endpoints are tested using tools such as **Postman** or **curl**. This includes verifying request and response formats, authentication using API keys, pagination behavior, and proper handling of success and error cases across all order-related operations.
2. **Workflow Testing** is conducted to validate the **end-to-end order processing flow** orchestrated by AWS Step Functions. This testing ensures that orders progress correctly through validation, payment processing, inventory updates, and notification steps, including correct branching behavior for different payment statuses and proper execution of error-handling paths.
3. **Event Testing** covers both **scheduled events** and **custom events** configured in Amazon EventBridge. Scheduled rules are tested to confirm that reports and inventory checks run at the expected times, while custom events are validated to ensure that order status changes correctly trigger downstream actions such as notifications.
4. For **Frontend Testing**, all user interface features are verified, including the dashboard, order creation form, order management table, reports section, and system status monitoring. This ensures correct data rendering, user interactions, and seamless communication with backend APIs.
5. Finally, **Integration Testing** validates that all services—API Gateway, Lambda functions, Step Functions, EventBridge, SNS, RDS, and the Amplify-hosted frontend—work together as a cohesive system. This testing phase confirms data consistency, event flow, error handling, and overall system stability before production use.